

PartnerAPI

Risk Register, Change Management and Closure Note

Software Engineering Project

October 6, 2026

Contents

1	Risk Management	3
2	Risk Register	3
3	Top Three Risks – RMMM	3
3.1	R1 – Internal Integration Delay	3
3.2	R3 – Scope Expansion	4
3.3	R4 – Security Vulnerability	4
4	Change Management	4
5	Change Request CR-01	5
5.1	Capture	5
5.2	Understand Reason	5
5.3	Impact Analysis	5
5.4	Decision	5
5.5	Plan Update	5
5.6	Implementation and Verification	5
6	Change Request CR-02	6
7	Combined Schedule Impact	6
8	Scope Decision	6

9 Lessons Learned	7
10 Closure Criteria	7
11 Final Closure Statement	8

1 Risk Management

Risk exposure is calculated as:

$$\text{Risk Exposure} = \text{Probability} \times \text{Impact}$$

where probability and impact are scored using the project's approved scale.

2 Risk Register

Table 1: PartnerAPI Risk Register

ID	Risk	Probability	Impact	Exposure	Response
R1	Internal booking-system integration delay	High	High	High	Early interface validation and integration testing.
R2	Partner integration problems	Medium	High	Medium/High	Sandbox validation and onboarding checklist.
R3	Scope expansion through custom requests	High	High	High	Formal change control and protected API baseline.
R4	Security vulnerability	Low	Very High	High	Security testing, authentication controls and review.
R5	Documentation delay	Medium	Medium	Medium	Documentation developed alongside API implementation.

3 Top Three Risks – RMMM

3.1 R1 – Internal Integration Delay

Risk Reduction:

Validate API contracts and dependencies early.

Monitoring:

Track integration milestones and unresolved interface issues.

Management:

Escalate blockers and prioritise integration work.

Mitigation:

Use mocks or stubs where appropriate during early development.

3.2 R3 – Scope Expansion

Risk Reduction:

Maintain a defined standard API scope.

Monitoring:

Review all change requests through the change log.

Management:

Require impact analysis before approval.

Mitigation:

Defer partner-specific features to a later release.

3.3 R4 – Security Vulnerability

Risk Reduction:

Use secure authentication, TLS and access controls.

Monitoring:

Review security test results and API logs.

Management:

Block release of critical unresolved security issues.

Mitigation:

Perform security remediation and regression testing.

4 Change Management

Every change shall pass through six stages:

1. Capture
2. Understand reason
3. Analyse impact
4. Decide

5. Update plan
6. Implement and verify

5 Change Request CR-01

Request: Partner-specific feature.

Effort: 12 days.

5.1 Capture

The partner request is recorded as CR-01.

5.2 Understand Reason

The requesting partner requires functionality that is not currently required by the general partner population.

5.3 Impact Analysis

Additional effort:

12 days

Potential impacts include development, testing, documentation and maintenance.

5.4 Decision

The feature is deferred from the standard initial release unless additional resources and schedule are formally approved.

5.5 Plan Update

The request is recorded in the backlog for future evaluation.

5.6 Implementation and Verification

If approved in a future release, the feature shall be implemented, tested and validated with the requesting partner.

6 Change Request CR-02

Request: Second partner-specific feature.

Effort: 18 days.

The same six-stage process shall be followed.

7 Combined Schedule Impact

The combined effort of the two custom requests is:

$$12 + 18 = 30 \text{ days}$$

The baseline project estimate is:

$$41.84 \text{ days}$$

Therefore, the new estimated duration becomes:

$$41.84 + 30 = 71.84 \text{ days}$$

Assuming five working days per week:

$$\frac{71.84}{5} = 14.37 \text{ weeks}$$

The resulting delay against the original 10-week target is:

$$14.37 - 10 = 4.37 \text{ weeks}$$

Therefore, accepting both requests without additional schedule or parallel capacity would jeopardise the 10-week launch objective.

8 Scope Decision

The standard PartnerAPI release shall remain focused on capabilities required by the partner ecosystem.

The two one-off features are therefore not automatically included in the baseline release.

This protects:

- Schedule

- API consistency
- Maintainability
- Testability
- Quality

9 Lessons Learned

The project should capture the following lessons:

1. Partner commitments should be validated before public launch promises.
2. A standard API contract reduces long-term integration complexity.
3. Custom requests require formal impact analysis.
4. Security and rate-limit requirements should be defined early.
5. API documentation should be developed alongside implementation.
6. Sandbox testing reduces partner integration risk.
7. Estimates should be treated as uncertain planning values rather than promises.

10 Closure Criteria

The project can be closed when:

- Required API functionality is delivered.
- First five partners are successfully onboarded.
- Quality targets are satisfied.
- Required documentation is complete.
- Critical defects are resolved or formally accepted.
- Risks are reviewed.
- Outstanding changes are transferred to the future-release backlog.

11 Final Closure Statement

The baseline PartnerAPI project is estimated at approximately **8.37 weeks** under the supplied sequential assumptions and therefore fits within the **10-week target**.

The two custom requests would increase the sequential estimate to approximately **14.37 weeks**.

Consequently, the standard API scope should be protected and the custom requests managed separately through formal change control.

This approach allows the project team to maintain schedule discipline, API consistency, maintainability and overall release quality.